



This short tutorial on kernel programming will set you on a journey to doing bigger things.

The core and central component of an operating system is known as the kernel. All the basic hardware level functions like task management and disk management are taken care of by the kernel of an operating system. When a user-level process requires direct access to a hardware component, it makes a request to the kernel, which is referred to as a system call.

The kernel resides in a protected area, accessible only to authorised users. It can directly access the hardware and therefore kernel functions are generally faster than normal user functions. The main functions of the kernel are to access all computer resources, manage all resources, as well as manage the memory and the entire device.

The kernel determines who can access it. It has something called 'rings' (as shown in Figure 1), which help it determine who has access to it and how. Basically, there is a specific region for

kernel-level programs to run, known as a kernel space. Similarly, there is a space where user programs run, called the user space.

But why should we learn kernel-level programming?

The main benefit is that by programming at the kernel level, we gain access to and control all the basic subsystems including system management, networking, memory,

and file handling. We can develop modules for the kernel separately and incorporate them into it while the machine is still running. This means we can use the kernel ourselves, bypassing user layers to access it.

From Figure 1, it is clear that the kernel level is the innermost and the most protected layer, while the application layer is the outermost and the least protected layer within the hardware.

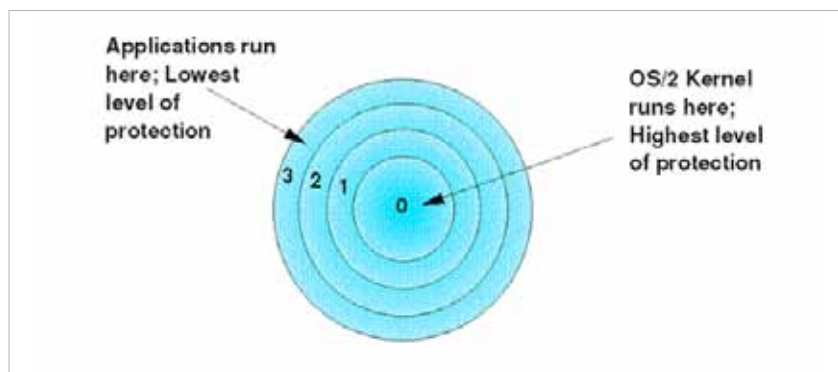


Figure 1: Operating system architecture